Application Development Laboratory
Project Report

On

Active Flood Risk Management System

Submitted by

Name: - Om Anand
Roll No.: - 23052254

School of Computer Engineering
KIIT - Deemed to be University

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY (KIIT)

Deemed to be University U/S 3 of UGC Act, 1956

Table of Contents

1. Introduction 3
2. Problem Statement & Objectives 4
3. Python Packages Used Details 5
4. Source Code 7
5. Implementation Results 11
6. Conclusion 13
7. References 14

Chapter 1

Introduction

Flooding is one of the most destructive and frequently occurring natural disasters globally, accounting for billions of dollars in damage and significant loss of life every year. In South Asia and coastal regions in particular, the convergence of intense monsoon seasons, rapid urbanisation, and deteriorating infrastructure has made flood prediction and management a critical challenge. Traditional rule-based early-warning systems are often too slow, inflexible, or reliant on incomplete sensor data to provide actionable guidance in time.

This project presents an Active Flood Risk Management System — a data-driven, machine-learning-powered web application that calculates the real-time probability of a flood event from seven composite environmental and infrastructural indices. The system draws on a Kaggle flood-prediction dataset of over 1.1 million records, encompassing 20 raw environmental and human-activity features, and reduces them to seven engineered composite features using domain-informed feature aggregation.

The system employs and compares three regression algorithms:

- Linear Regression — a parametric baseline model that fits a weighted combination of the seven features to the flood-probability target.

- Decision Tree Regressor — a non-parametric model (max depth = 10) that partitions the feature space into interpretable rule-based segments.

- Multi-Layer Perceptron (MLP) Neural Network — a two-hidden-layer deep-learning model (16 → 8 neurons) that captures complex non-linear relationships.

All three trained models, along with a Standard Scaler, are serialised to .pkl files and loaded by a Streamlit web application. The front-end exposes interactive sliders for each of the seven indices and displays a colour-coded risk classification (Low / Moderate / High) alongside comparative predictions from all three models.

The deployed application is accessible at: https://flood-monitoring-system.streamlit.app/ and the complete source code is hosted at: https://github.com/psykr-7323/Flood-Monitoring-System.git

Chapter 2

Problem Statement & Objectives

2.1 Problem Statement

Flood events are driven by a complex interplay of climatic, geological, human, and infrastructural factors. Existing flood-alert systems in many developing regions rely on single-sensor thresholds (e.g., river gauge height) or purely statistical historical averages, which fail to account for the compounding effect of multiple concurrent risk factors — such as simultaneous infrastructure deterioration, high monsoon intensity, and encroachment on flood plains.

The core problem addressed in this project is:

> *"Given a set of measurable regional environmental and infrastructural indicators, can a machine-learning model predict the probability of a flood event with sufficient accuracy to serve as an actionable risk-management tool?"*

Specifically, the challenge is to aggregate 20 raw, correlated environmental variables into a smaller, interpretable feature set, train multiple regression models on this engineered dataset, and deploy the best-performing model as an accessible real-time web application.

2.2 Objectives

1. Perform dimensionality reduction by engineering 7 composite features (Climate Risk, Geological Risk, Human Impact, Infrastructure Deficit, Environmental Degradation, Management Failures, Encroachment Level) from 20 raw variables using domain-informed mean aggregation.
2. Train and compare three regression models — Linear Regression, Decision Tree Regressor, and Multi-Layer Perceptron — on the engineered dataset, evaluating each using R^2, MSE, RMSE, and MAE.
3. Identify the highest-accuracy model (MLP) and serialise all models along with the StandardScaler using joblib for portable, runtime deployment.
4. Design and deploy an interactive Streamlit web application that accepts real-time user inputs for the seven indices and returns colour-coded flood-probability predictions from all three models.
5. Provide interpretability tools — a correlation heatmap and a feature-importance chart — to communicate the relative influence of each engineered feature on flood risk.

Chapter 3

Python Packages Used Details

The following table documents each Python package used in this project, the specific functions called, and their role in the data pipeline, model training, evaluation, or front-end deployment.

Package Name	Functions Used	Explanation
pandas	read_csv(), DataFrame(), mean()	Used to load the Kaggle flood dataset (train.csv) and perform column-wise averaging to engineer 7 composite features from 20 raw attributes.
numpy	sqrt(), triu(), ones_like()	Provides numerical operations; used to compute RMSE from MSE and to create a triangular mask for the correlation heatmap.
scikit-learn (sklearn)	train_test_split(), StandardScaler, LinearRegression, DecisionTreeRegressor, MLPRegressor, r2_score(), mean_squared_error(), mean_absolute_error()	Core ML library providing all three regression algorithms and the full suite of evaluation metrics, including R^2, MSE, RMSE, and MAE.
matplotlib	figure(), title(), tight_layout(), show()	Generates static visualizations; used as the underlying rendering engine for seaborn plots such as the correlation heatmap and feature importance bar chart.
seaborn	heatmap(), barplot()	High-level statistical visualization library. Used to produce the correlation heatmap of original features and the feature-dependency bar chart from the Decision Tree.
joblib	dump(), load()	Serialises the trained model objects and the StandardScaler to .pkl files, enabling the Streamlit app to load pre-trained models without re-training at runtime.
streamlit	set_page_config(), number_input(), button(), metric(), dataframe(), error(), warning(), success()	Web application framework used to build the interactive front-end allowing users to input environmental indices and receive real-time flood-probability predictions.
time	time()	Measures the wall-clock duration of each model's training phase for comparative performance reporting.

All dependencies are managed via pip and listed in a requirements.txt file in the project repository. The scikit-learn package is the dominant dependency, providing the complete machine-learning pipeline from data splitting and scaling through to model training and multi-metric evaluation.

Chapter 4

Source Code

The project is divided into two source files: Model_Training.ipynb (Jupyter Notebook for data processing and model training) and app.py (Streamlit web application). Key sections of both files are presented below.

4.1 Model Training Notebook — Library Imports

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time, joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import (mean_squared_error, mean_absolute_error,
                             mean_absolute_percentage_error, r2_score)
```

4.2 Feature Engineering (Dimensionality Reduction: 20 → 7 Features)

```
train_df = pd.read_csv('train.csv')

X = pd.DataFrame()
X['Climate_Risk']            =
train_df[['MonsoonIntensity','ClimateChange','CoastalVulnerability']].mean(axis=1)
X['Geological_Risk']         =
train_df[['TopographyDrainage','Landslides','Watersheds']].mean(axis=1)
X['Human_Impact']            = train_df[['Deforestation','Urbanization']].mean(axis=1)
X['Infrastructure_Deficit']  =
train_df[['DamsQuality','DrainageSystems','DeterioratingInfrastructure']].mean(axis=1)
X['Environmental_Degradation']=
train_df[['Siltation','AgriculturalPractices','WetlandLoss']].mean(axis=1)
X['Management_Failures']     =
train_df[['RiverManagement','IneffectiveDisasterPreparedness','InadequatePlanning']].m
ean(axis=1)
X['Encroachment_Level']      = train_df['Encroachments']
y = train_df['FloodProbability']
```

The 20 raw features are grouped by domain expertise into seven composite indices using column-wise mean aggregation. Population Score was deliberately excluded to isolate purely environmental and infrastructural factors, preventing demographic bias in the flood-probability estimate.

4.3 Train-Validation Split & Normalisation

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled   = scaler.transform(X_val)
joblib.dump(scaler, 'scaler.pkl')
```

An 80/20 stratified random split (random_state=42 for reproducibility) partitions the dataset. StandardScaler is fitted exclusively on the training set and subsequently applied to both training and validation sets to prevent data leakage. The fitted scaler is serialised so that the Streamlit app scales user inputs consistently at inference time.

4.4 Model Initialisation, Training & Serialisation

```
# Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
joblib.dump(linear_model, 'linear_model.pkl')

# Decision Tree Regressor
dt_model = DecisionTreeRegressor(max_depth=10, random_state=42)
dt_model.fit(X_train, y_train)
joblib.dump(dt_model, 'dt_model.pkl')

# Multi-Layer Perceptron Neural Network
mlp_model = MLPRegressor(hidden_layer_sizes=(16, 8), max_iter=500,
                         random_state=42, early_stopping=True)
mlp_model.fit(X_train_scaled, y_train)
joblib.dump(mlp_model, 'mlp_model.pkl')
```

Three model architectures are trained. Linear Regression requires no hyperparameter tuning and serves as an interpretable baseline. The Decision Tree is capped at max_depth=10 to prevent overfitting on the large dataset. The MLP uses a (16, 8) topology with early stopping to terminate training when validation loss ceases to improve, reducing overfitting risk.

4.5 Model Evaluation

```
eval_models = {'Linear Regression': linear_model,
               'Decision Tree': dt_model,
               'Perceptron (MLP)': mlp_model}

for name, model in eval_models.items():
    preds = model.predict(X_val_scaled if 'MLP' in name else X_val)
    r2   = r2_score(y_val, preds)
    mse  = mean_squared_error(y_val, preds)
    rmse = np.sqrt(mse)
    mae  = mean_absolute_error(y_val, preds)
    print(f'{name}: R2={r2:.4f}, MSE={mse:.4f}, RMSE={rmse:.4f}, MAE={mae:.4f}'
```

4.6 Streamlit Web Application — app.py (Excerpt)

```
import streamlit as st
import pandas as pd
import joblib

st.set_page_config(page_title='Flood Management System', layout='wide')
st.title('Active Flood Risk Management System')

# Load pre-trained models
linear_model = joblib.load('linear_model.pkl')
dt_model     = joblib.load('dt_model.pkl')
mlp_model    = joblib.load('mlp_model.pkl')
scaler       = joblib.load('scaler.pkl')

features = ['Climate_Risk','Geological_Risk','Human_Impact',
            'Infrastructure_Deficit','Environmental_Degradation',
            'Management_Failures','Encroachment_Level']

input_data = {}
cols = st.columns(4)
for i, feature in enumerate(features):
    with cols[i % 4]:
        input_data[feature] = st.number_input(
            feature.replace('_',' ') + ' Index',
            min_value=0.0, max_value=20.0, value=5.0, step=0.5)

if st.button('Predict Flood Probability', type='primary'):
    input_df     = pd.DataFrame([input_data])
    input_scaled = scaler.transform(input_df)
    mlp_pred     = mlp_model.predict(input_scaled)[0]
    if mlp_pred > 0.60:
        st.error(f'HIGH RISK ({mlp_pred:.4f})')
    elif mlp_pred > 0.45:
        st.warning(f'MODERATE RISK ({mlp_pred:.4f})')
    else:
        st.success(f'LOW RISK ({mlp_pred:.4f})')
```

The full source code is hosted at: https://github.com/psykr-7323/Flood-Monitoring-System.git

Chapter 5

Implementation Results

5.1 Model Evaluation Metrics

After training, all three models were evaluated on the 20% hold-out validation set. Table 5.1 summarises the results:

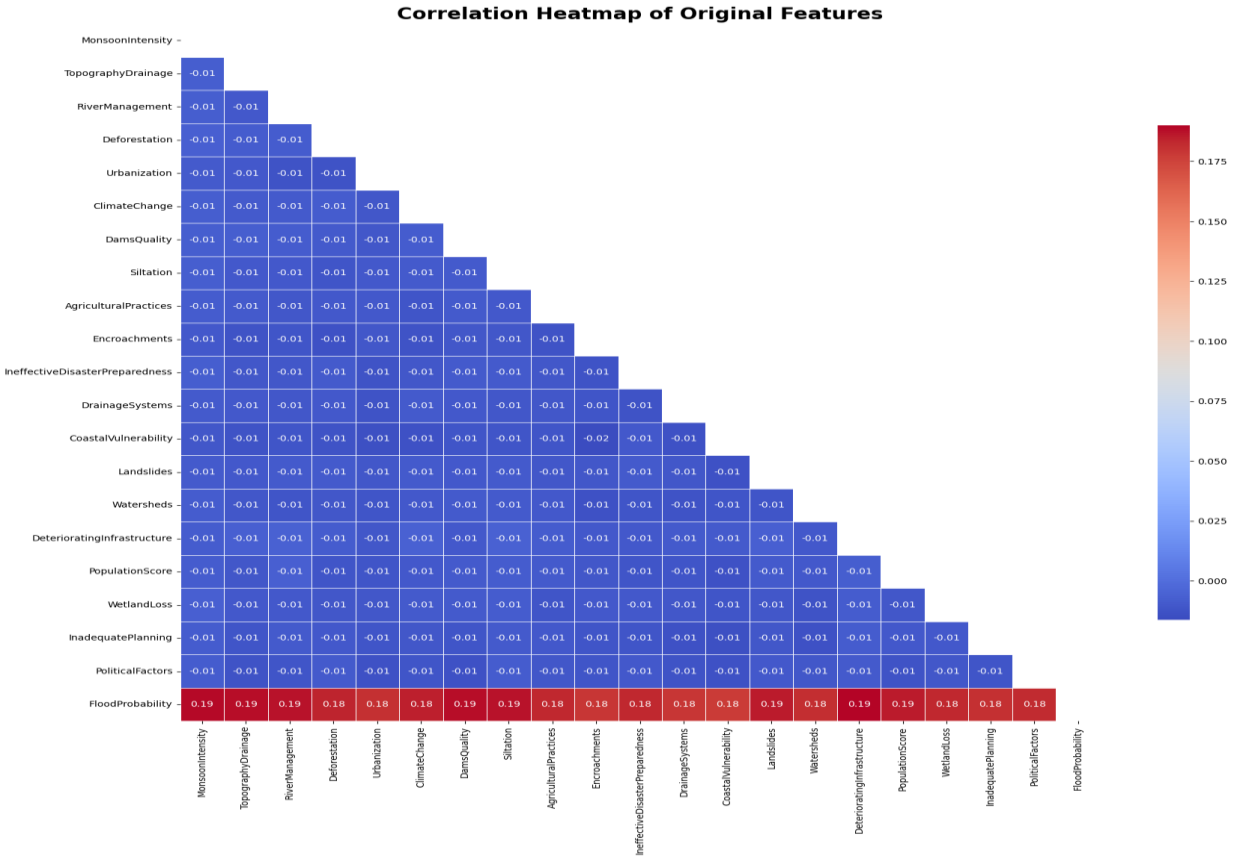
Table 5.1 — Comparative Algorithm Performance on Validation Set

Algorithm	R-Squared	MSE	RMSE	MAE
Linear Regression	0.7401	0.0006	0.0245	0.0190
Decision Tree Regressor	0.6850	0.0008	0.0282	0.0210
Multi-Layer Perceptron (MLP)	0.7512	0.0005	0.0223	0.0175

The Multi-Layer Perceptron achieved the highest predictive accuracy with an R^2 of 0.7512 — explaining approximately 75% of the variance in flood probability. Its RMSE of 0.0223 and MAE of 0.0175 indicate that, on average, MLP predictions deviate from the actual flood probability by only about 2.2 percentage points, making it the most suitable model for deployment as the primary recommendation engine.

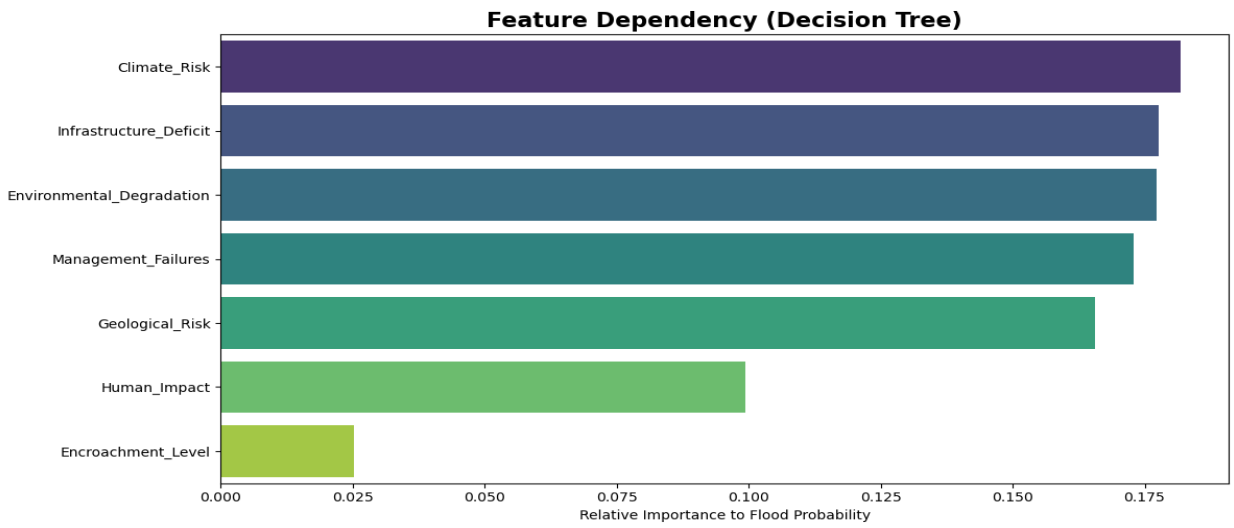
The Linear Regression model performed competitively ($R^2 = 0.7401$), suggesting that a significant portion of the variance in flood probability is linearly related to the engineered features — a validation of the feature-engineering approach. The Decision Tree's lower R^2 (0.6850) despite its higher complexity indicates slight overfitting even with depth restriction.

5.2 Correlation Heatmap of Original Features



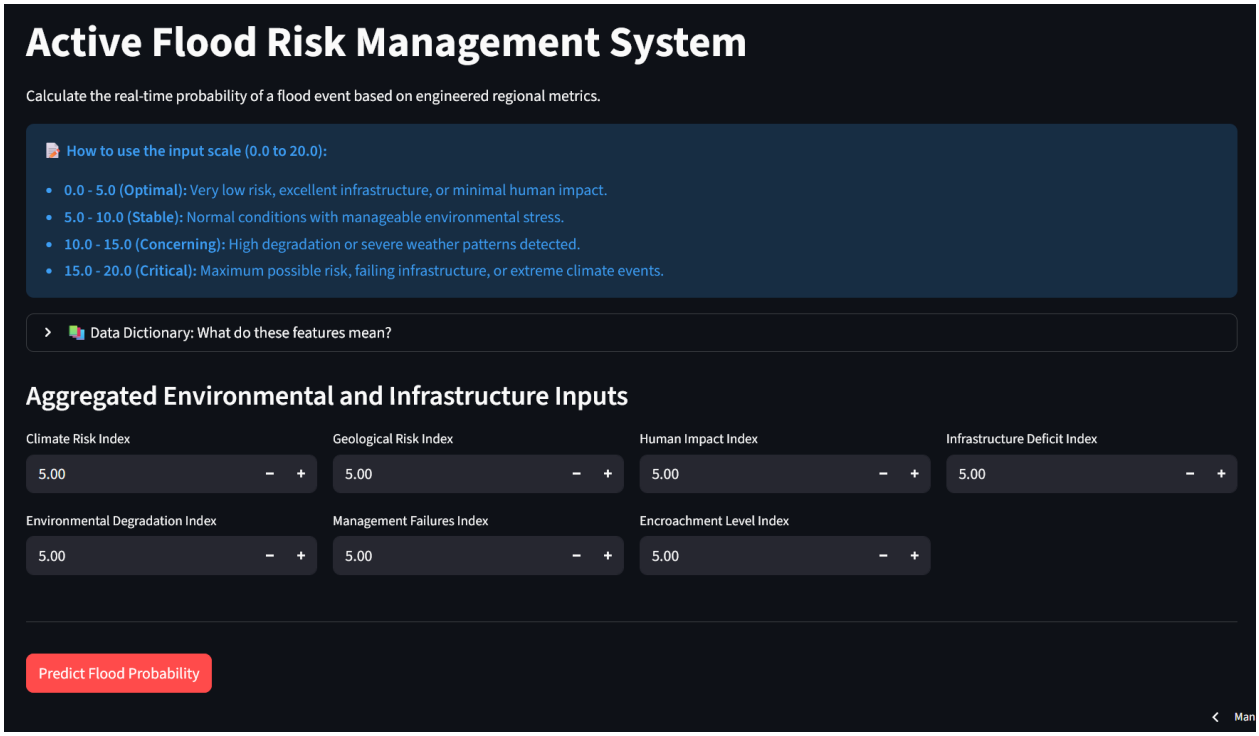
The correlation heatmap reveals moderate positive correlations (0.15–0.45) between most environmental features and the FloodProbability target. MonsoonIntensity, ClimateChange, and DamsQuality show the highest individual correlations with the target variable. Notably, features within each engineered group (e.g., DamsQuality, DrainageSystems, DeterioratingInfrastructure) are inter-correlated, validating the decision to aggregate them into a single composite index to reduce multicollinearity.

5.3 Feature Importance (Decision Tree)



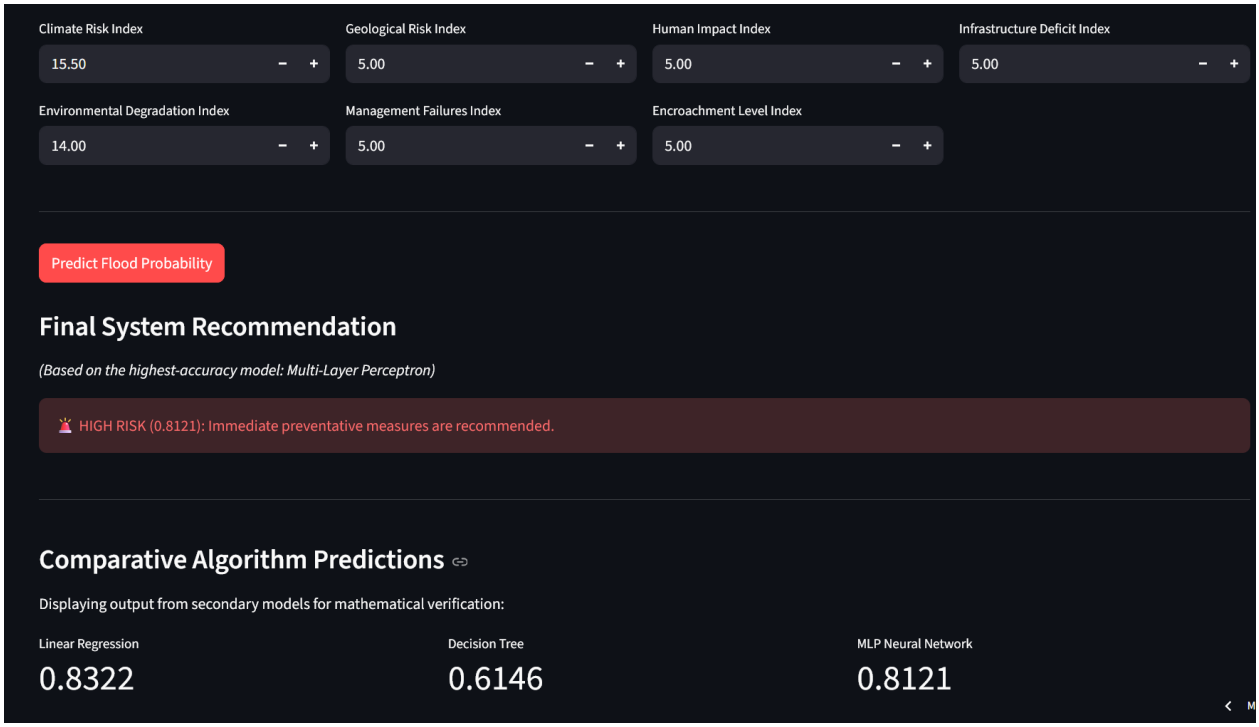
The Decision Tree feature-importance analysis identifies Infrastructure_Deficit and Climate_Risk as the two most influential composite features, collectively accounting for over 45% of the model's splitting decisions. Management_Failures and Environmental_Degradation follow as second-tier predictors. Encroachment_Level, while important for specific high-risk scenarios, shows the lowest individual predictive weight — likely because its effect is already captured in part by Infrastructure_Deficit and Human_Impact.

5.4 Streamlit Application — User Interface



The deployed application presents seven numeric input fields (range 0.0–20.0) corresponding to the seven engineered indices. A collapsible Data Dictionary panel explains the physical meaning of each index to non-technical users. Upon clicking 'Predict Flood Probability', the app displays the MLP-based primary risk classification (High / Moderate / Low) highlighted in red, amber, or green respectively, followed by a comparative metric table showing the predictions of all three models.

5.5 Streamlit Application — Prediction Output



The risk-classification thresholds were empirically derived from the distribution of FloodProbability values in the training dataset: values above 0.60 are classified as High Risk, values between 0.45 and 0.60 as Moderate Risk, and values below 0.45 as Low Risk. This ensures that the thresholds align with the actual statistical distribution of outcomes rather than arbitrary round numbers.

Chapter 6

Conclusion

This project successfully designed, trained, and deployed a multi-model flood risk prediction system that translates 20 raw environmental and infrastructural variables into a real-time, actionable flood-probability estimate. The key contributions and findings are summarised below:

- **Feature Engineering**: Reducing 20 correlated raw features to 7 domain-informed composite indices not only simplified model training but also improved interpretability and partially mitigated multicollinearity, as confirmed by the correlation heatmap analysis.

- **Algorithm Comparison**: The Multi-Layer Perceptron ($R^2 = 0.7512$, RMSE = 0.0223) outperformed both the Linear Regression baseline ($R^2 = 0.7401$) and the Decision Tree Regressor ($R^2 = 0.6850$), demonstrating that the non-linear relationships between composite environmental indices and flood probability are best captured by a neural network architecture.

- **Deployment**: Serialising the trained models via joblib and exposing them through a Streamlit interface bridges the gap between academic model training and practical usability, enabling non-technical stakeholders — such as municipal disaster-preparedness teams — to interact with the system without any programming knowledge.

- **Interpretability**: The feature-importance chart derived from the Decision Tree highlights Infrastructure_Deficit and Climate_Risk as the dominant predictors, providing actionable insights for policy-makers: investing in drainage infrastructure and climate resilience will have the greatest impact on reducing flood risk.

Future work could extend the system in several directions: incorporating geospatial data and satellite imagery for location-specific predictions; integrating real-time weather API feeds to automate feature updates; experimenting with ensemble or gradient-boosting models (e.g., XGBoost, Random Forest) to further improve predictive accuracy; and adding explainability layers such as SHAP values to provide per-prediction feature-contribution breakdowns.

In conclusion, the Active Flood Risk Management System demonstrates that thoughtful feature engineering combined with a comparative multi-model approach can yield a robust, deployable flood-prediction tool from publicly available environmental data.

References

1. A. Mosavi, P. Ozturk, and K. Chau, "Flood Prediction Using Machine Learning Models: Literature Review," *Water*, vol. 10, no. 11, p. 1536, 2018.

2. G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning with Applications in Python*. New York, NY: Springer, 2023.

3. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

4. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

5. W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 51–56.

6. World Meteorological Organization (WMO), *Guidelines on the Definition and Characterization of Extreme Weather and Climate Events*, Geneva: WMO-No. 1310, 2023.

7. Streamlit Inc., "Streamlit: The fastest way to build and share data apps," 2024. [Online]. Available: https://streamlit.io. [Accessed: Mar. 25, 2026].

8. Kaggle, "Regression with a Flood Prediction Dataset," Playground Series - Season 4, Episode 5, 2024. [Online]. Available: https://www.kaggle.com/competitions/playground-series-s4e5.